
UTM

UNIVERSITI TEKNOLOGI MALAYSIA

BACHELOR OF COMPUTER SCIENCE (DATA ENGINEERING) WITH HONS.

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

**SECP3133-02 - HIGH PERFORMANCE DATA PROCESSING**

---

**PROJECT 2:**

# Real-Time Sentiment Analysis using Apache Spark and Kafka

---

GROUP : SPARKLING SPARKMIND

**PREPARED BY :**


| NAME | MATRIC NO |
|---|---|
| SAFIYA NURSYAHADAH BINTI MASNOOR | A23CS0176 |
| AIN NURNABILA BINTI MOHD AZHAR | A23CS0207 |
| FARRA NURZAHIN BINTI ZAHARIL ANUAR | A23CS0079 |
| DAYANG FARAH FARZANA BINTI ABANG IDHAM | A23CS0071 |


**PREPARED FOR:**

DR. SEAH CHOON SEN

## 1.0 Introduction

This section introduces the project by outlining the motivation for real-time sentiment analysis, the specific objectives the group set out to achieve, and the boundaries of what the project does and does not cover. Together, these three parts frame the rest of the report. Section 2 onward describes how the project actually executed against the scope defined here.

### 1.1 Background

Online platforms generate customer opinions faster than any team can read them manually, and businesses increasingly need a way to understand how customers feel about their products the moment feedback is posted rather than days later. Real-time sentiment analysis addresses this gap by combining natural language processing with streaming data infrastructure, allowing organisations to detect shifts in customer satisfaction as they happen instead of relying on periodic manual reviews. Prior work analysing Shopee's Malaysian user base has similarly found that customer sentiment on e-commerce platforms tends to be dominated by clearly polarised opinion rather than neutral commentary (Saad et al., 2023).

This project was undertaken as part of SECP3133, High Performance Data Processing, which requires students to move beyond theoretical data processing concepts and build a working, end-to-end system. The group selected Shopee app reviews as the subject of analysis, since e-commerce feedback is text-rich, publicly relevant to Malaysian consumers, and naturally suited to a three-class sentiment problem (positive, negative, neutral). Building this pipeline gave the group hands-on exposure to the practical challenges of combining Apache Kafka, Apache Spark, and Elasticsearch, tools that are widely used in industry for high-throughput data engineering.

### 1.2 Objectives

The project was guided by the following objectives:

1. To design and implement a real-time streaming pipeline using Apache Kafka and Apache Spark Structured Streaming capable of ingesting and classifying text data continuously.
2. To train and evaluate at least two sentiment classification models, a Naive Bayes classifier and an LSTM neural network, on a labelled dataset of Shopee app reviews.

3. To integrate a trained model into the Spark streaming job so that incoming reviews are classified as positive, negative, or neutral automatically.
4. To store the classified output in Elasticsearch and present the results through an interactive dashboard.
5. To compare the performance of batch processing against streaming processing across metrics such as throughput, latency, and time-to-first-output.

### 1.3 Project Scope

This project focuses on Shopee app reviews as the Malaysian-relevant data source, in line with the assignment brief's requirement to process text data from a Malaysian e-commerce or social platform, and consistent with prior Malaysian-context studies of the same platform (Saad et al., 2023). The dataset consists of 1,707 labelled reviews, classified into three sentiment classes: positive, negative, and neutral.

The scope covers the full pipeline lifecycle: data cleaning and preprocessing, training and comparison of two sentiment models (Naive Bayes and LSTM), streaming ingestion through Kafka, real-time inference through Spark Structured Streaming, storage in Elasticsearch, and visualization through a dashboard. The project also includes a structured comparison between batch and streaming processing modes, as required by the brief.

The scope does not extend to live collection from Shopee's platform in real time (the dataset was collected and labelled beforehand, then replayed through Kafka to simulate a live stream), nor does it include deployment on a multi-node cluster. The pipeline was built and tested on a single local machine using Docker.

## 2.0 Data Acquisition and Preprocessing

This section describes the process of collecting and preparing the text data used in the sentiment analysis pipeline. It covers the selection of a Malaysian-relevant data source, the tools and methods used for data collection, the natural language processing (NLP) techniques applied to clean and normalise the raw text and a summary of the final dataset produced.

### 2.1 Data Source

For this project, the selected Malaysian-relevant data source is the Shopee Malaysia application on the Google Play Store. Shopee is one of the most widely used e-commerce platforms in Malaysia, with millions of active users generating a continuous stream of user-generated reviews. The platform serves as a rich and representative source of public sentiment from Malaysian consumers, covering a broad range of topics including product quality, delivery experience, customer service and overall application usability.

#### 2.1.1 Justification

The selection of Shopee Malaysia Google Play reviews as the primary data source was based on the following considerations :

- **Malaysian relevance**: Shopee Malaysia is one of the leading e-commerce platforms in the country, with a large and active Malaysian user base. The reviews reflect authentic sentiments from local users, making the dataset contextually appropriate for this project.
- **Availability and accessibility**: Google Play reviews are publicly accessible and do not require API authentication or special permissions, which simplifies the data collection process significantly.
- **Natural sentiment diversity**: User reviews inherently contain a wide spectrum of sentiments from highly positive reviews expressing satisfaction to strongly negative reviews expressing frustration, making them ideal for multi-class sentiment classification.
- **Volume**: The Shopee Malaysia application contains tens of thousands of English-language reviews, ensuring that a sufficiently large and representative dataset can be collected.

- **Pre-existing rating labels**: Each review on Google Play is accompanied by a star rating (1 to 5), which can be directly mapped to sentiment labels without the need for manual annotation.

### 2.1.2 Collection Method and Tool

Data collection was performed using the google-playscraper Python library, which provides a programmatic interface to retrieve reviews directly from the Google Play Store without requiring any API key or authentication credentials. The library supports batch retrieval with a continuation token mechanism, enabling the systematic collection of large volumes of reviews across multiple requests. The collection process was configured in Table 2.1 below:

Table 2.1 : Configuration Parameters for Google Play Review Collection

| **Parameter** | **Value** |
|---|---|
| Application Package ID | com.shopee.my |
| Language | English (en) |
| Country | Malaysia (my) |
| Sort Order | Newest first |
| Batch Size | 200 reviews per request |
| Target Volume | 3,000 reviews |

The scraper was executed in Google Colaboratory with a one-second delay between each batch request to avoid rate limiting. A continuation token was passed between requests to ensure sequential and non-overlapping retrieval of reviews.

### 2.1.3 Volume of Data Collected

A total of 3,000 raw reviews were successfully collected from the Shopee Malaysia Google Play listing. After the preprocessing pipeline was applied, the final cleaned dataset contained 1,707 usable records. The reduction in volume is attributed to the removal of null entries, duplicate reviews, non-informative short texts and reviews that yielded fewer than three tokens after cleaning.

## 2.2 Preprocessing Steps

Raw user-generated text data from Google Play contains significant amounts of noise that can negatively affect model performance if left unaddressed. A structured NLP preprocessing pipeline was implemented to clean and normalise the raw reviews before they were passed to the model training phase. The pipeline consists of the following steps, applied sequentially.

**Step 1 : Removal of Null and Duplicate Entries**

Reviews with missing (NaN) or empty ("") text content were identified and removed from the dataset. Additionally, duplicate reviews were detected using the review_id field which is a unique identifier assigned by the Google Play Store to each review and dropped to prevent data leakage during model training and evaluation. Figure 2.1 below shows the code snippet used to handle missing values.

```
df.dropna(subset=['review_text'], inplace=True)
df = df[df['review_text'].str.strip() != '']  # Remove empty strings
df.drop_duplicates(subset='review_id', inplace=True)
after = len(df)
```

Figure 2.1 : Remove null and duplicate reviews

**Step 2 : Lowercase Conversion**

All review text was converted to lowercase to ensure consistency and to prevent the same word appearing in different cases (e.g., "Good", "good", and "GOOD") from being treated as distinct tokens during the vectorisation process. For example:

- Before: “This App is AMAZING! Best shopping experience.”
- After: “this app is amazing! best shopping experience.”

**Step 3 : Noise Removal**

Several categories of non-informative content were removed using regular expressions:

- **URLs**: Hyperlinks beginning with http, https, or www were removed.
- **Email addresses**: Any text matching an email pattern was removed.
- **Mentions and hashtag**s: Tokens beginning with @ or # were removed.
- **Digits**: All numerical characters were removed, as standalone numbers do not carry semantic meaning for sentiment classification.
- **Special characters**: All non-alphabetic characters, including punctuation, emojis, and symbols, were removed, retaining only lowercase alphabetic characters and whitespace.

For example:

- Before: “visit shopee.com or call 012-3456789 @shopeemy #shopeemalaysia”
- After: “visit or call”

**Step 4 : Tokenisation**

The cleaned text was tokenised into individual words using the word_tokenize function from the Natural Language Toolkit (NLTK). Tokenisation splits continuous text into discrete units (tokens) that can be processed individually in subsequent steps. For example:

- Before: “this app is very slow and always crashing”
- After: [‘this’, ‘app’, ‘is’, ‘very’, ‘slow’, ‘and’, ‘always’, ‘crashing’]

**Step 5 : Stopword Removal**

Common English stopwords were removed using NLTK's built-in English stopword list, which includes words such as "the", "is", "and", "a", and "of". These words appear at high frequency across all sentiment classes and do not contribute meaningful discriminative signals for classification.

In addition to the standard NLTK list, Figure 2.2 shows a set of custom domain-specific stopwords was defined and applied to remove terms that are ubiquitous in app review contexts and therefore non-discriminative across sentiment classes:

```
# Add custom stopwords relevant to app reviews
custom_stopwords = {'app', 'shopee', 'please', 'really', 'would', 'also',
                    'get', 'use', 'used', 'using', 'one', 'like', 'even',
                    'still', 'much', 'many', 'every', 'could', 'well'}
```

Figure 2.2 : Custom Stopwords

Tokens with fewer than three characters were also discarded at this stage to remove uninformative short tokens such as "ok", "la", and single letters.

**Step 6 : Lemmatisation**

Each remaining token was reduced to its canonical base form (lemma) using NLTK's WordNetLemmatizer. Lemmatisation groups different inflected forms of the same word under a single representation, reducing the effective vocabulary size and improving model generalisation.

- “running” → “run”
- “crashed” → “crash”
- “deliveries” → “delivery”

**Step 7: Removal of Short Reviews**

Following the full pipeline, each cleaned review's token count was calculated. Reviews containing fewer than three tokens after all cleaning steps were removed, as they do not contain sufficient textual content to be informative for model training.

### 2.2.1 Before and After Preprocessing Examples

The following examples illustrate the effect of the complete preprocessing pipeline on raw review text across all three sentiment classes:

Table 2.2 : Before and After Preprocessing

| **Sentiment** | **Before Preprocessing** | **After Preprocessing** |
|---|---|---|
| Positive | "I love this app! Very fast delivery and great customer service 😊👍" | "love fast delivery great customer service" |
| Negative | "Terrible!! My order was cancelled 3 times and no refund yet @shopeemy" | "terrible order cancelled time refund" |
| Neutral | "It's okay I guess. Delivery was slow but the seller was quite responsive." | "okay delivery slow seller responsive" |

## 2.3 Final Dataset Summary

This section presents a comprehensive summary of the dataset produced following the completion of the data acquisition and preprocessing pipeline. It outlines the key attributes of the final dataset, the sentiment label mapping applied, the class distribution across the three sentiment categories and sample records drawn from a cleaned output file.

### 2.3.1 Dataset Overview

Table 2.3 summarizes the key attributes of the final dataset produced at the conclusion of the data acquisition and preprocessing phase. It captures essential details including the data source, collection tool and environment, as well as the volume of records before and after preprocessing.

Table 2.3 : Dataset Overview

| Attribute | Value |
|---|---|
| Data Source | Shopee Malaysia - Google Play Store |
| Application Package ID | com.shopee.my |
| Collection Tool | google-play-scraper (Python) |
| Collection Environment | Google Colaboratory |
| Raw Records Collected | 3,000 |
| Records After Preprocessing | 1,707 |
| Input Feature Column | cleaned_text |
| Target Label Column | sentiment |
| Number of Sentiment Classes | 3 (positive, neutral, negative) |
| Output File (raw) | data/raw_data/raw_reviews.csv |
| Output File (cleaned) | data/cleaned_data.csv |

### 2.3.2 Sentiment Label Mapping

Sentiment labels were derived from the Google Play star ratings included with each review using the following mapping:

- Positive (4-5 stars) → User is satisfied with the product or service
- Neutral (3 stars) → User has a mixed or indifferent experience
- Negative (1-2 stars) → User dissatisfied with the product or service

This mapping is consistent with widely adopted conventions in review-based sentiment analysis research and does not require any manual labelling effort.

### 2.3.3 Sample Records from Cleaned Dataset

Figure 2.3 presents a selection of sample records drawn from the final cleaned dataset. Each record displays the original review text alongside its processed counterpart, demonstrating the

effect of the cleaning pipeline on real user-generated content. Figure 2.3 also includes the corresponding star rating, sentiment label, review date and token count for each record.

| review_id | review_text | cleaned_text | star_rating | sentiment | review_date | token_count |
|---|---|---|---|---|---|---|
| d006c8e9-db89-463 | The price is mislead | price misleading sho | 1 | negative | 2026-06-26 9:17:43 | 19 |
| 9714ab55-df50-42db | shopee Food so bad | food bad driver wait | 1 | negative | 2026-06-26 9:13:22 | 24 |
| 3289def2-fbcf-4750- | Please keep up the g | keep good work than | 5 | positive | 2026-06-26 9:01:31 | 4 |
| 8b499a43-c3f0-4dfd | Shopping made easi | shopping made easi | 5 | positive | 2026-06-26 8:47:46 | 10 |
| a0d7a36e-daa3-4c2 | become more comp | become complicate | 2 | negative | 2026-06-26 8:47:14 | 9 |
| b9b10213-c10d-4e9 | terbaik Dan senang c | terbaik dan senang d | 5 | positive | 2026-06-26 8:46:05 | 4 |
| 515f6327-24fc-48c9 | Excellent service.. K | excellent service kee | 5 | positive | 2026-06-26 8:24:00 | 3 |
| aeb27b29-1111-4c2 | Saya suka shopping | saya suka shopping k | 5 | positive | 2026-06-26 8:09:52 | 8 |
| cd398675-88f6-403a | good selalu belanja | good selalu belanja | 5 | positive | 2026-06-26 7:33:05 | 7 |
| 0129c61c-48a9-437 | merumitkan maca pe | merumitkan maca pe | 1 | negative | 2026-06-26 5:50:02 | 3 |
| eaabfa59-eca5-4bd5 | don't straightaway go | dont straightaway go | 3 | neutral | 2026-06-26 5:25:36 | 6 |
| 3abbe588-ef9e-4d7a | Greenpeace! messy | greenpeace messy w | 1 | negative | 2026-06-26 4:41:08 | 17 |
| 68f91179-597a-4c6c | Shopee offers a con | offer convenient eco | 5 | positive | 2026-06-26 4:38:43 | 24 |
| bfd73d34-2d48-4159 | really2 bad. it shows | bad show discounted | 1 | negative | 2026-06-26 3:23:35 | 17 |
| 3b195195-ae61-487 | WE CAN BUY ANYTI | buy anything want | 5 | positive | 2026-06-26 2:58:17 | 3 |
| abed0ec4-8f38-4b9c | how they treat their l | treat loyal customer | 1 | negative | 2026-06-26 1:05:18 | 27 |
| 6f771e4e-7062-4eaf | VIP subscription is a | vip subscription scan | 1 | negative | 2026-06-25 23:47:58 | 25 |
| 65ecc337-043c-438 | Shopping sini mmg t | shopping sini mmg te | 1 | negative | 2026-06-25 23:41:06 | 35 |
| f17fb543-70e8-4971 | Aplikasi Shopee sang | aplikasi sangat muda | 5 | positive | 2026-06-25 23:35:04 | 51 |
| 0cca63e9-5a01-4d5 | Stop spamming your | stop spamming user | 1 | negative | 2026-06-25 23:32:00 | 11 |
| 9f6308a0-f710-47fc | Hi, could you please | remove feature autol | 1 | negative | 2026-06-25 21:03:51 | 16 |
| 29a17976-ebda-466 | I know e-commerce a | know ecommerce ap | 3 | neutral | 2026-06-25 20:52:23 | 24 |
| 769c72cc-846b-43f2 | very low price...good | low pricegood qualit | 5 | positive | 2026-06-25 14:48:23 | 4 |
| 7062f5ed-d7b6-4152 | Hate low volume live | hate low volume live | 1 | negative | 2026-06-25 14:16:10 | 7 |
| 75df89ba-9eb4-41db | Excellent shopping a | excellent shopping a | 5 | positive | 2026-06-25 13:28:02 | 7 |
| e33968c2-af69-4ad1 | Shopee is a very cor | convenient online sh | 5 | positive | 2026-06-25 13:00:39 | 12 |

Figure 2.3 : Sample Records from the Cleaned Dataset (cleaned_data.csv)

## 3.0 Sentiment Model Development

This section describes the development, training, and evaluation of two sentiment classification models built to classify Shopee app reviews as positive, negative, or neutral.

### 3.1 Model Selection

Two sentiment classification models were developed and compared in this project: a Naive Bayes classifier (Machine Learning) and a Long Short-Term Memory (LSTM) network (Deep Learning). These two approaches were selected because they represent fundamentally different paradigms for text classification, enabling a meaningful comparison of model complexity, training cost, and predictive performance.

#### 3.1.1 Model 1 - Naive Bayes (MultinomialNB with TF-IDF)

Naive Bayes is a probabilistic classifier based on Bayes' theorem. It assumes that each feature (in this case, each word) contributes independently to the predicted class. Despite this simplifying assumption, Naive Bayes performs surprisingly well on text classification tasks and is widely used as a strong, efficient baseline. Text was represented using Term Frequency-Inverse Document Frequency (TF-IDF) vectorization (Pedregosa et al., 2011), which assigns higher weights to words that are distinctive to a document relative to the rest of the corpus. The vocabulary was capped at the 5,000 most frequent terms to control dimensionality.

Because scikit-learn's MultinomialNB does not support a ‘class_weight’ parameter directly, balanced sample weights were computed using ‘compute_sample_weight(class_weight='balanced')’ and passed via the ‘sample_weight’ argument during training. This ensures that the minority neutral class (only 67 records, 3.9% of the dataset) contributes proportionally to the decision boundary, rather than being overwhelmed by the majority positive and negative classes.

#### 3.1.2 Model 2 - LSTM (Long Short-Term Memory Neural Network)

LSTM is a type of recurrent neural network specifically designed to capture sequential dependencies in text (Hochreiter & Schmidhuber, 1997). Unlike Naive Bayes, which treats text as a bag of words with no word order, an LSTM reads the review token by token and

maintains a hidden state that carries contextual information forward. This makes it better suited to detecting sentiment expressed through word order or multi-word phrases (e.g., "not good" vs "good").

The architecture consisted of three layers: an Embedding layer that maps each word integer ID to a 64-dimensional dense vector, a single LSTM layer with 64 hidden units, and a Dense output layer with 3 units and softmax activation for 3-class probability output. To address class imbalance, ‘compute_class_weight('balanced')’ was used to derive per-class weights which were passed via the ‘class_weight’ argument in ‘model.fit()’, penalising misclassifications of the neutral class more heavily during backpropagation.

### 3.2 Training Process

The dataset consisted of 1,707 Shopee app reviews labelled as positive (949 records, 55.6%), negative (691 records, 40.5%), or neutral (67 records, 3.9%). All three classes were retained in accordance with the project brief's requirement for positive/negative/neutral classification output in the streaming pipeline.

The dataset was divided into three subsets using stratified splitting to preserve class proportions across all partitions as shown in Table 3.1.

Table 3.1: Dataset division

| Split | Proportion | Approximate Size |
|---|---|---|
| Training | 70% | 1,194 records |
| Testing | 20% | 342 records |
| Validation | 10% | 171 records |

Stratified splitting was used (via ‘stratify=y’ in scikit-learn's ‘train_test_split’) to ensure that even the small neutral class was represented in every subset. A fixed random seed of 42 was applied throughout to ensure reproducibility.

### 3.3 Evaluation and Comparison

Both models were evaluated on the same held-out test set (20%, ~342 records) using four standard metrics: accuracy, precision, recall, and F1-score. The same 'evaluate_model()' function was used for both models to ensure a consistent and fair comparison.

The Naive Bayes classifier achieved an overall accuracy of 76.90% as shown in Figure 3.1, performing strongly on the positive class (F1 = 0.88) and reasonably well on the negative class (F1 = 0.77). However, as shown in Figure 1, the model completely failed to identify the neutral class, scoring a precision, recall and F1-score of 0.00. This means the model never predicted neutral for any test record in which all 14 true neutral samples were misclassified as either negative or positive. This outcome is attributable to the severe class imbalance in the dataset, where neutral accounts for only 3.9% of records (67 samples). Even with balanced sample weighting applied during training, the volume of neutral training examples was too small for the model to learn a reliable decision boundary for that class.

```
===== Naive Bayes (Test Set) =====
Accuracy: 0.7690

Classification Report:
              precision    recall  f1-score   support

    negative       0.80      0.74      0.77       138
     neutral       0.00      0.00      0.00        14
    positive       0.93      0.85      0.88       190

    accuracy                           0.77       342
   macro avg       0.57      0.53      0.55       342
weighted avg       0.84      0.77      0.80       342
```

Figure 3.1: Naive Bayes confusion matrix

The LSTM achieved an overall accuracy of 50.00%, which is notably lower than the Naive Bayes model. Examining the per-class results in Figure 3.2 reveals the cause: the LSTM's neutral class recall reached 1.00 (it correctly retrieved all 14 true neutral samples) but its precision collapsed to just 0.08, meaning it incorrectly predicted neutral for a large number of negative and positive records as well. This is a symptom of the 'class_weight='balanced'' strategy overcompensating for the extreme scarcity of neutral samples which the model learned to over-predict the neutral class to minimise the heavily penalised neutral errors, at the cost of misclassifying the majority of negative records (recall

= 0.21). In effect, the class weighting pushed the LSTM toward a trade-off that hurt overall accuracy significantly.

```
===== LSTM (Test Set) =====
Accuracy: 0.5000

Classification Report:
              precision    recall  f1-score   support

    negative       0.91      0.21      0.34       138
     neutral       0.08      1.00      0.15        14
    positive       0.98      0.67      0.80       190

    accuracy                           0.50       342
   macro avg       0.65      0.63      0.43       342
weighted avg       0.91      0.50      0.59       342
```

Figure 3.2 LSTM Confusion Matrix

**Side-by-Side Comparison**

Table 3.2 shows the comparison between both models. Bold values indicate the better-performing model for each metric.

Table 3.2: Model Comparison

| Metric | Naive Bayes | LSTM |
|---|---|---|
| Overall Accuracy | **0.7690** | 0.5000 |
| Weighted Precision | **0.84** | 0.91 |
| Weighted Recall | **0.77** | 0.50 |
| Weighted F1-Score | **0.80** | 0.59 |
| Negative F1 | **0.77** | 0.34 |
| Neutral F1 | 0.00 | 0.15 |

| | | |
|---|---|---|
| Positive F1 | **0.88** | 0.80 |
| Training Time | Near-instant | ~45 seconds (10 epochs) |

## 4.0 Apache System Architecture

### 4.1 Pipeline Overview

The system is a real-time streaming pipeline that carries each Shopee app review from ingestion through to dashboard visualisation with no manual steps in between. It is built from four layers, each detailed further in Sections 4.2 to 4.4:

1. **Ingestion.** A Kafka producer publishes each review as a JSON message to the sentiment-input topic, simulating a live stream.
2. **Stream Processing.** A Spark Structured Streaming job consumes the topic in micro-batches and classifies each review using a broadcast Naive Bayes model.
3. **Storage.** Classified records are written to an Elasticsearch index, with deterministic document IDs to keep writes idempotent.
4. **Visualisation.** Kibana queries that index to render the real-time dashboard covered in Section 5.

Since each layer is connected only through the Kafka topic and the Elasticsearch index, the producer, the Spark job, and the dashboard can each run, fail, and restart independently rather than as one monolithic script. All components were containerised or run locally on a single Windows 11 machine, with Docker Compose provisioning Kafka and ZooKeeper (Figure 4.2).

Figure 4.1 is the system architecture of the real-time sentiment analysis pipeline, from raw review ingestion through to dashboard visualisation where it traces the path of a single review through the pipeline: it enters as a Kafka message, is classified by the Spark streaming job, lands in Elasticsearch, and finally surfaces on the Kibana dashboard. Colour groups the four layers described, with the boundary between Kafka's blue and Spark's teal marking the transition from ingestion to inference.

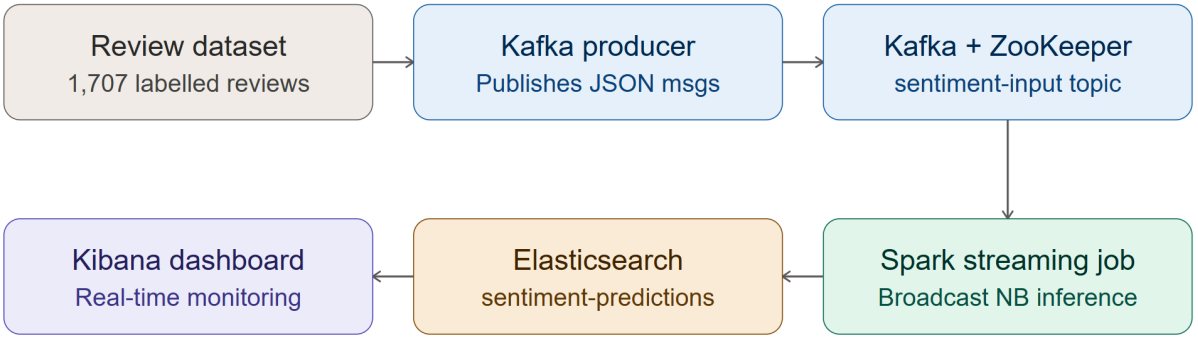


Figure 4.1: End-to-end system architecture

### 4.2 Kafka Configuration

The message broker was provisioned using Confluent Apache Kafka version 7.5.0 (Kreps et al., 2011). ZooKeeper handles broker coordination and leader election for the Kafka cluster. The Docker Compose configuration used to provision the infrastructure is shown in Figure 4.2 below.

```
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    container_name: zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFER_MAX_MS_ANNOUNCE: 1000
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Figure 4.2: Docker containers for Kafka and ZooKeeper running successfully

The Kafka producer publishes to the topic sentiment-input on port 9092. Each message is serialised as a UTF-8 encoded JSON payload containing the fields review_id, review_text, cleaned_text, true_sentiment, star_rating, review_date, and source. The producer was configured with acks='all' to require full broker acknowledgement before confirming each

publish operation, and retries=3 to handle transient network faults. An intentional 20-millisecond inter-message delay was applied via time.sleep (0.02) to simulate a live e-commerce review stream. As shown in Figure 4.3, all 1,707 reviews were successfully published to the topic.

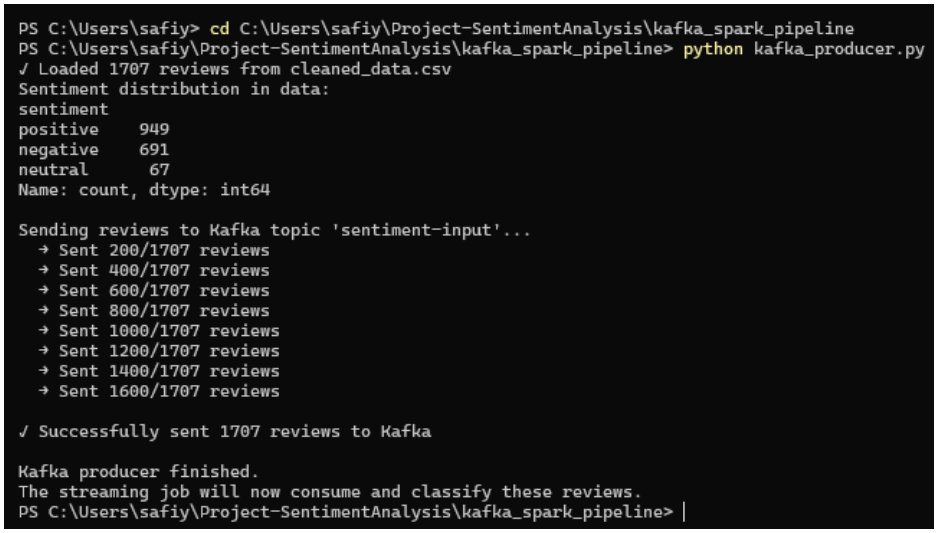


Figure 4.3: Kafka producer output confirming successful delivery of 1707 reviews

### 4.3 Spark Structured Streaming Job

The stream processing layer was implemented using Apache Spark 4.1.1 Structured Streaming, submitted via spark-submit with the Kafka connector package spark-sql-kafka-0-10_2.13:4.1.1. The Spark session was configured with spark.streaming.kafka.maxRatePerPartition set to 500, which caps the number of records consumed per Kafka partition per micro-batch to prevent resource exhaustion during sustained ingestion.

Rather than relying on runtime schema inference, the pipeline enforces a static schema binding using a StructType definition as shown in Figure 4.4 below. This eliminates type ambiguity and ensures deterministic field mapping across all streaming micro-batches.

```
kafka_schema = StructType([
    StructField("review_id", StringType()),
    StructField("review_text", StringType()),
    StructField("cleaned_text", StringType()),
    StructField("true_sentiment", StringType()),  # Ground truth
    StructField("star_rating", LongType()),
    StructField("review_date", StringType()),
    StructField("source", StringType())
])
```

Figure 4.4: PySpark StructType Schema Definition for Kafka Ingestion

The pre-trained Multinomial Naive Bayes model and TF-IDF vectorizer, serialised as naive_bayes_model.pkl, are loaded once at driver initialisation and distributed to all executor nodes using Spark Broadcast Variables. This caches the model parameters in worker memory as shown in Figure 4.5, eliminating repeated serialisation overhead across micro-batches.

```
# Broadcast models to workers
nb_model_broadcast = spark.sparkContext.broadcast((nb_model, tfidf_vectorizer))
label_map_broadcast = spark.sparkContext.broadcast(label_to_sentiment)
```

Figure 4.5: Initialisation and Broadcasting of Naive Bayes Model and Mapping Variables

Sentiment inference is performed through two User-Defined Functions (UDFs) which predict_naive_bayes returns the predicted sentiment label (negative, neutral, or positive), while predict_naive_bayes_confidence returns the maximum class probability as a confidence score. Both UDFs call the broadcast model internally, keeping inference self-contained within the Spark executor. Each processed record is enriched with nb_sentiment, nb_confidence, processed_at timestamp, and is_correct_nb, a boolean field comparing the predicted label against the ground truth true_sentiment column. As shown in Figure 4.6, the streaming job successfully processed all records across 14 micro-batches (Batch 0 through Batch 13).

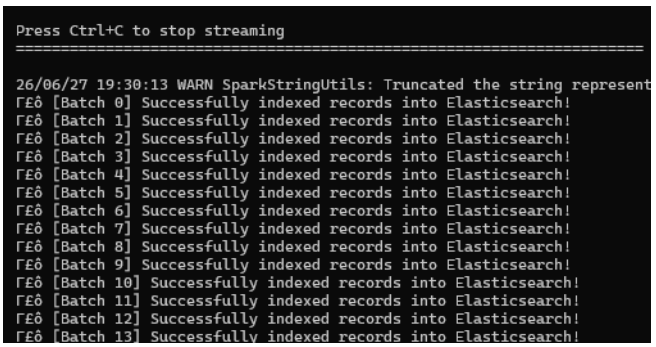


Figure 4.6: Live Terminal Output Showing Successful Elasticsearch Indexing Across Micro-Batches

### 4.4 Storage Layer

Processed prediction records are written to the Elasticsearch index sentiment-predictions using a custom foreachBatch sink handler implemented entirely in Python using the requests library. This approach was adopted because native Spark-Elasticsearch connector JARs are incompatible with the Spark 4.x runtime on Windows 11, producing driver-level initialisation faults. By bypassing the native JAR dependency entirely, the pipeline achieves Elasticsearch integration through direct HTTP POST requests to the Bulk API endpoint at http://localhost:9200/_bulk.

Within each micro-batch invocation, the output DataFrame is serialised to JSON strings via toJSON().collect() and formatted as Newline-Delimited JSON (NDJSON) conforming to the Elasticsearch Bulk API specification. As illustrated in Figure 4.7, the payload alternates between a metadata action line specifying the target index and document identifier, and a payload data line containing the actual record attributes (data_dict).

Each document is explicitly assigned a deterministic identifier derived from review_id. By incorporating this unique ID directly into the bulk action header, the system ensures strict processing idempotency; any duplicated records resulting from unexpected micro-batch retries or stream replays will safely overwrite the existing entries rather than generating duplicate documents within the index.

```
bulk_data += json.dumps({"index": {"_index": "sentiment-predictions", "_id": review_id}}) + "\n"
# Payload data line
bulk_data += json.dumps(data_dict) + "\n"
```

Figure 4.7: NDJSON Formatting for Elasticsearch Bulk API Payload Construction

Successful end-to-end indexing was verified by querying the Elasticsearch document count endpoint, which returned the following response confirming that all 1,707 records were persisted with no data loss as shown in Figure 4.8.

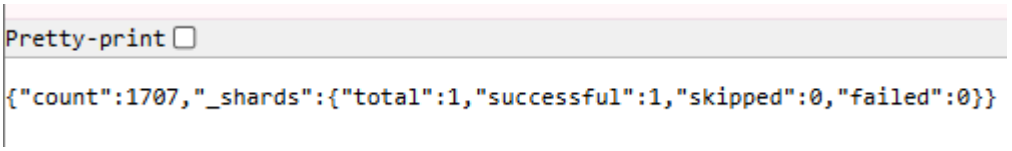


Figure 4.8: Elasticsearch document count confirming successful indexing of all 1,707 records

## 5.0 Analysis and Results

With the models trained and the streaming pipeline operating end-to-end, this section turns to what the classified data actually shows. It covers the overall sentiment breakdown across the 1,707 processed reviews, how sentiment moves over the review period, the language patterns that separate positive from negative feedback, and how these are surfaced through the dashboard for at-a-glance monitoring.

### 5.1 Sentiment Distribution

The full dataset of 1,707 Shopee app reviews is heavily imbalanced across the three sentiment classes: 949 reviews (55.6%) are positive, 691 reviews (40.5%) are negative, and only 67 reviews (3.9%) are neutral, as shown in Figure 5.1.

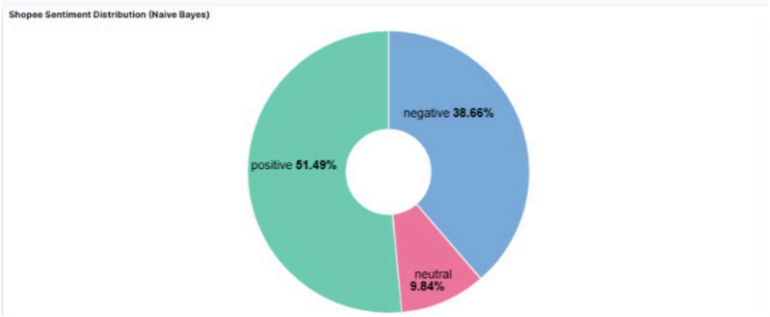


***Figure 5.1*** *Sentiment Distribution Pie*

This imbalance is the most important characteristic of the dataset and directly explains the model behaviour discussed in Section 3.3. Because neutral reviews make up less than 4% of the data, both the Naive Bayes and LSTM models struggled to learn a reliable decision boundary for that class: Naive Bayes ignored it almost entirely, while the LSTM over-corrected and over-predicted it. In practice, this reflects a common pattern in customer review data, where users tend to leave clearly positive or clearly negative feedback, with neutral or mixed opinions rarely expressed in writing. This pattern is consistent with other Naive Bayes studies conducted on Shopee review data, which report similarly skewed positive/negative splits and the same difficulty isolating a minority neutral class (Pratmanto et al., 2020). This is worth noting as a limitation of the dataset rather than the pipeline itself.

### 5.2 Sentiment Trends Over Time

The Model Confidence Score Trend panel (Figure 5.2) plots the average prediction confidence for each sentiment class against processing time, bucketed into 30-minute intervals.

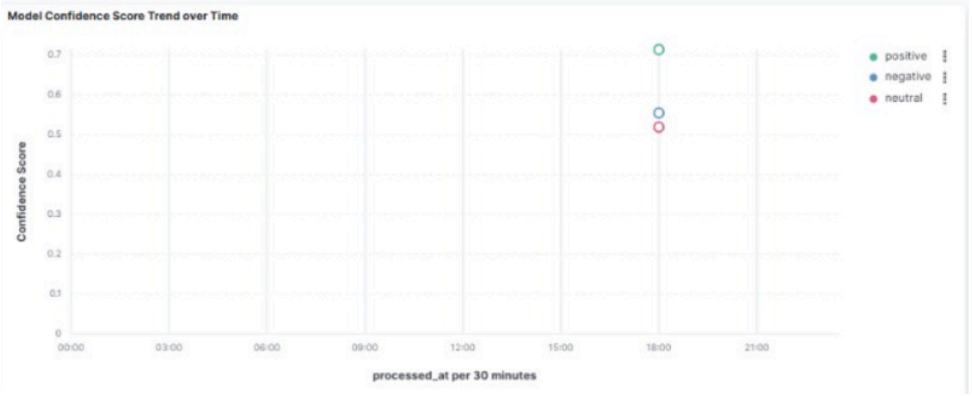


***Figure 5.2*** *Model Confidence Score Trend over Time chart*

In practice, the chart shows one tight cluster of points sitting around 18:00, with the rest of the timeline empty. This is expected: the full 1,707-review dataset was streamed through Kafka in roughly 34 seconds, as noted in Section 4.2, so the entire run landed inside a single 30-minute bucket rather than spreading across the day. Within that one window, positive predictions carried the highest average confidence (around 0.70), followed by negative (around 0.55) and neutral (around 0.50), the model is least sure of itself exactly on the class it has the least training data for. Because this run was a one-off replay rather than a continuous live feed, there isn't a real multi-hour trend to interpret yet. This panel would be far more informative if the pipeline ran against reviews arriving naturally over hours or days.

### 5.3 Word Clouds

The final dashboard build did not include a dedicated word cloud panel. Instead, the team built a Real-Time Review Logs & Model Predictions table (Figure 5.3), which lists individual reviews next to their true label and the model's prediction, and serves a similar purpose: showing exactly where the model's reasoning breaks down at the text level.

***Figure 5.3*** *Real-Time Review Logs & Model Predictions table*

One example from the log stands out: a review that translates roughly to "cheaper" can't be used at the shop was labelled negative but predicted positive. This is a fairly typical Naive Bayes failure, the word "murah" (cheap) is a strong positive signal on its own, and since the model treats text as a bag of words with no sense of order or context, it missed that the review was actually a complaint. Other reviews in the log, such as clear complaints about customer service or clear praise for the shopping experience, were classified correctly, which fits the pattern already noted in Section 3.1.2: Naive Bayes does well when sentiment words are unambiguous, and struggles when a single strong word overrides the review's real meaning.

### 5.4 Dashboard Overview

The dashboard was built in Kibana and combines the three panels above into a single view (Figure 5.4).


***Figure 5.4** full dashboard screenshot*

The sentiment distribution donut gives an at-a-glance read of the overall class split. The confidence trend chart tracks how sure the model was of its predictions per class over time, which becomes genuinely useful once the pipeline runs continuously instead of as a single batch replay. The review log table lets a viewer scan individual misclassifications directly, while an aggregated "Other" row collapses the bulk of correctly and incorrectly classified reviews by combination (for example, 780 reviews correctly predicted positive, versus 48 positive reviews predicted negative and 42 predicted neutral), so the full 1,707-row detail doesn't have to be scrolled through by hand.

## 6.0 Optimization and Comparison

### 6.1 Pipeline Optimization

Several design decisions were made during the development of the streaming pipeline to improve reliability and performance on local Windows hardware.

The first optimization involved replacing the native Spark-Elasticsearch JAR connector with a custom Python REST handler. Under Spark 4.1.1 on Windows 11, the conventional elasticsearch-hadoop connector JAR produced driver compatibility errors during initialisation. This was resolved by implementing a pure-Python foreachBatch sink that serialises each micro-batch into NDJSON and pushes it directly to the Elasticsearch Bulk API over HTTP, eliminating any JAR dependency entirely.

The second optimization addressed checkpoint directory path handling on Windows. The default POSIX-style path /tmp/checkpoint_es caused silent OS-level directory mapping failures on Windows, which corrupted the streaming checkpoint state. This was resolved by specifying an absolute Windows raw string path, as shown in Figure 6.1 below.

```
try:
    # Use a localized Windows directory path for checkpointing to prevent OS path errors
    query = predictions_df \
        .writeStream \
        .foreachBatch(send_batch_to_es) \
        .option("checkpointLocation", r"C:\Users\safiy\Project-SentimentAnalysis\kafka_spark_pipeline\checkpoint_es") \
        .start()
```

Figure 6.1: Configuring an Absolute Windows Path for Local Spark Checkpointing

The third optimization involved using Spark Broadcast Variables to cache the Naive Bayes model and TF-IDF vectorizer in executor memory. Without broadcasting, Spark would serialise and transmit the model object to workers on every micro-batch, causing significant CPU overhead. Broadcasting ensures the model is loaded once and reused across all batches, which is particularly important given that the pickle file carries a cross-version warning between scikit-learn 1.6.1 (training environment) and 1.9.0 (inference environment).

The fourth optimization was the enforcement of a static Kafka schema using StructType. Dynamic schema inference at runtime introduces unnecessary overhead and risks type mismatches on malformed messages. Declaring the schema explicitly ensures consistent parsing across all 1,707 records and across all 14 micro-batches processed during the run.

Table 6.1 summarises the pipeline optimizations applied and the issues they resolved.

Table 6.1: Pipeline Optimization Summary

| Issue Encountered | Cause | Optimization Applied |
|---|---|---|
| ES connector JAR failure | Spark 4.x incompatibility on Windows | Custom Python REST handler via foreachBatch |
| Checkpoint directory fault | POSIX path invalid on Windows filesystem | Absolute Windows raw string path |
| Model serialisation overhead per batch | Broadcast not used | Spark Broadcast Variables for NB model and vectorizer |
| Schema inference overhead | Dynamic type resolution at runtime | Static StructType schema declaration |

### 6.2 Batch vs Streaming Performance Comparison

The streaming pipeline processed all 1,707 Shopee reviews across 14 micro-batches. The Kafka producer completed publishing at a rate of approximately one review per 20 milliseconds, yielding a total producer runtime of roughly 34 seconds for the full dataset. The Spark streaming job consumed and classified records continuously as they arrived, with each micro-batch successfully indexed into Elasticsearch as confirmed by the batch success logs.

In comparison, a batch processing approach would load the entire 1,707-record dataset into memory at once, apply TF-IDF vectorization and Naive Bayes prediction in a single pass, and write all results simultaneously. While batch mode offers lower infrastructure overhead and faster raw processing time on a single machine, it produces results only after the full dataset is loaded and cannot provide any intermediate output during execution.

Table 6.2 presents a structured comparison between the two processing paradigms as applied to this pipeline.

Table 6.2: Batch vs Streaming Performance Comparison

| Metric | Batch Processing | Streaming Processing |
|---|---|---|

| | | |
|---|---|---|
| Records processed | 1707 | 1707 |
| Processing trigger | Manual, on-demand | Continuous, event-driven |
| Time-to-first output | After full dataset loads | Per micro-batch (~Batch 0 onward) |
| Producer delay required | No | Yes (20ms per record) |
| Infrastructure overhead | Low — single Python process | Higher — Kafka + Spark + ES stack |
| Fault recovery | Full re-run on failure | Checkpoint-based incremental recovery |
| Data completeness verified | - | {"count":1707} from Elasticsearch |
| Suitable for real-time monitoring | No | Yes |

The key trade-off is clear batch processing is more resource-efficient for one-off historical analysis, while streaming is better suited for operational scenarios where sentiment needs to be monitored continuously as reviews arrive. In this project, the streaming approach is justified by the goal of real-time sentiment monitoring, and the successful indexing of all 1,707 records into Elasticsearch with zero data loss confirms that the pipeline operated correctly throughout.

## 7.0 Conclusion and Future Work

This project successfully delivered a working end-to-end sentiment analysis pipeline for Shopee app reviews, combining Apache Kafka for streaming ingestion, Apache Spark Structured Streaming for real-time classification, and Elasticsearch for storage and querying. All 1,707 reviews were published through Kafka and processed across 14 micro-batches with zero data loss, confirming that the pipeline operated reliably from end to end.

Two sentiment models were trained and compared: a Naive Bayes classifier and an LSTM network. Naive Bayes achieved a higher overall accuracy (76.9% vs. 50.0%) and was ultimately the model integrated into the streaming pipeline. However, neither model handled the neutral class well: Naive Bayes failed to predict it at all, while the LSTM over-predicted it at the cost of accuracy elsewhere. This outcome highlights a limitation of the dataset rather than the modelling approach, since neutral reviews make up under 4% of the data and there simply wasn't enough signal for either model to learn that class reliably.

A few limitations are worth noting. First, the dataset size (1,707 records) is relatively small for training a deep learning model like LSTM, which typically benefits from much larger volumes of labelled text. Second, the pipeline was deployed on a single local machine rather than a distributed cluster, so the performance figures reported here reflect a single-node setup rather than production-scale throughput. Third, the custom Python REST sink used to write to Elasticsearch, while functional, is less efficient than a native Spark-Elasticsearch connector. Related work applying attention-based LSTM architectures to Shopee review data has shown that richer model designs can partially offset this kind of class imbalance (Ng et al., 2022), which supports the future work direction outlined below.

For future work, three improvements would meaningfully strengthen the system:

1. Address class imbalance at the data level. Collecting or oversampling additional neutral-class reviews would likely improve both models' ability to distinguish that class, rather than relying solely on class-weighting during training.
2. Test a transformer-based model. A fine-tuned DistilBERT or similar model may capture contextual sentiment cues (e.g., sarcasm, mixed reviews) that neither Naive Bayes nor a single-layer LSTM (Hochreiter & Schmidhuber, 1997) can.

3. Move to a distributed deployment. Running Spark and Kafka across multiple nodes (e.g., via Databricks or AWS EMR) would allow the batch-vs-streaming comparison to be re-evaluated at a scale closer to real production workloads.

References

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735 to 1780. https://doi.org/10.1162/neco.1997.9.8.1735 (open-access copy: https://www.bioinf.jku.at/publications/older/2604.pdf)

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB Workshop* (pp. 1 to 7). https://notes.stephenholiday.com/Kafka.pdf *(this is a workshop paper and was never assigned a DOI; the link is the freely accessible PDF, confirmed live)*

Ng, H., Chia, G. J. W., Yap, T. T. V., Goh, V. T., Nguyen, H. D., Polignano, M., & Yadav, A. (2022). Modelling sentiments based on objectivity and subjectivity with self-attention mechanisms. *F1000Research*, *11*, 366. https://doi.org/10.12688/f1000research.73131.2

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825 to 2830. https://doi.org/10.5555/1953048.2078195 (open-access copy: https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf)

Pratmanto, D., Rousyati, R., Wati, F. F., Widodo, A. E., Suleman, S., & Wijianto, R. (2020). App review sentiment analysis Shopee application in Google Play Store using Naive Bayes algorithm. *Journal of Physics: Conference Series*, *1641*, 012043. https://doi.org/10.1088/1742-6596/1641/1/012043

Saad, N. H. M., Sin, L. P., & Yaacob, Z. (2023). An exploratory of sentiment analysis on e-commerce business platform: Shopee Malaysia. *Jurnal Komunikasi: Malaysian Journal of Communication*, *39*(4), 63 to 82. https://doi.org/10.17576/JKMJC-2023-3904-04

Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster computing with working sets. In *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '10)*. https://doi.org/10.5555/1863103.1863113 (open-access copy: https://www.usenix.org/conference/hotcloud-10/spark-cluster-computing-working-sets)

The Apache Software Foundation. (n.d.). *Structured Streaming programming guide*. Apache Spark Documentation. https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

The Apache Software Foundation. (n.d.). *Apache Kafka documentation*. https://kafka.apache.org/documentation

Elastic. (n.d.). *Bulk API*. Elasticsearch Guide. https://www.elastic.co/guide/en/elasticsearch/reference/current/docs-bulk.html

Docker Inc. (n.d.). *Compose file reference*. Docker Documentation. https://docs.docker.com/compose/

TensorFlow. (n.d.). *Keras API reference*. https://www.tensorflow.org/api_docs